



SahelysAgents

Documentation Backend & IA — Phase 1

KIENDREBEOGO Moussa

18 août 2026

Table des matières

1	Sommaire	2
2	1. Vue d'ensemble du rôle backend	2
3	2. Stack technologique complète	2
4	3. Installation de l'environnement	3
4.1	3.1 Outils à installer sur le poste de travail	3
4.2	3.2 Installation des dépendances Python	3
4.3	3.3 Vérification côté serveur SAHELYS (à faire avec la direction/IT)	3
4.4	3.4 Fichier <code>.env</code> — un par service (jamais commité dans Git)	4
4.5	3.5 Lancer les deux services en développement	4
5	4. Architecture du système	4
6	5. Modèle de données	5
7	6. Sécurité et authentification	7
8	7. Endpoints à construire	7
8.1	Endpoints internes du service IA (port 8001, jamais exposés au frontend)	8
9	8. Plan de travail par sprints	8
9.1	Sprint 0 — Préparation de l'environnement	8
9.2	Sprint 1 — Squelette du projet et modèle de données	8
9.3	Sprint 2 — Authentification	9
9.4	Sprint 3 — Gestion des agents (CRUD)	9
9.5	Sprint 4 — Service IA séparé (Mistral/Ollama)	9
9.6	Sprint 5 — Intégration avec le module RAG (YANOGO Fabiana) + streaming . . .	10
9.7	Sprint 6 — Association de documents	10

9.8	Sprint 7 — Journalisation	10
9.9	Sprint 8 — Rôle administrateur central	10
9.10	Sprint 9 — Intégration avec l'équipe	11
9.11	Sprint 10 — Tests et préparation de la démonstration	11
10	9. État d'avancement global	11

1 Sommaire

1. Vue d'ensemble du rôle backend
 2. Stack technologique complète
 3. Installation de l'environnement
 4. Architecture du système
 5. Modèle de données
 6. Sécurité et authentification
 7. Endpoints à construire
 8. Plan de travail par sprints (suivi d'avancement)
 9. État d'avancement global (à cocher)
-

2 1. Vue d'ensemble du rôle backend

D'après le cahier des charges (§4), le bloc Backend & IA couvre :

- **Orchestrateur** : reçoit les requêtes du frontend, coordonne l'appel au module RAG et au modèle Mistral
- **Connexion au modèle IA** : Mistral via Ollama, exécuté sur le serveur SAHELYS (on-premise)
- **Authentification** : gestion des utilisateurs et des tokens JWT
- **Permissions** : contrôle d'accès par direction (RH, IT) et par rôle (collaborateur, référent)
- **Journalisation** : traçabilité des questions et réponses (§7)

Livrable officiel de fin de Phase 1 (§6) : « *API backend avec authentification et premier appel au modèle* ».

Décision : Backend et IA sont développés comme **deux services séparés** (deux applications FastAPI distinctes, deux ports), pour faciliter la maintenance et permettre un déploiement indépendant plus tard. Ce choix reste interne au développement backend/IA — invisible pour YANOGO Fabiana (RAG) et NANA Marc (frontend), qui continuent à consommer les mêmes contrats d'API qu'avant.

3 2. Stack technologique complète

Composant	Technologie	Rôle
Langage	Python 3.11+	Aligné avec le module RAG (YANOGO Fabiana)
Framework API	FastAPI	Deux applications séparées : service Backend (port 8000) et service IA (port 8001)
Serveur ASGI	Uvicorn	Fait tourner chacune des deux applications FastAPI
Base de données	PostgreSQL + pgvector	Tables relationnelles (agents, utilisateurs, logs) partagées avec le module RAG
ORM / accès DB	SQLAlchemy (+ asyncpg)	Manipuler la base depuis Python
Migrations	Alembic	Créer/faire évoluer les tables de façon versionnée
Authentification	PyJWT	Génération et vérification des tokens

Composant	Technologie	Rôle
Hachage mots de passe	passlib (bcrypt) ou pwdlib	Sécuriser les mots de passe stockés
Variables d'environnement	python-dotenv	Gérer les secrets (clé JWT, identifiants DB) hors du code
Appel au modèle IA	httpx (appel HTTP vers Ollama)	Communication avec Mistral
Tests	pytest	Vérifier chaque endpoint
Documentation API auto-générée	Swagger UI (inclus nativement dans FastAPI, /docs)	Tester les endpoints sans outil externe

4 3. Installation de l'environnement

4.1 3.1 Outils à installer sur le poste de travail

- ☐ Python 3.11 ou supérieur
- ☐ PostgreSQL (client `psql` ou pgAdmin pour se connecter au serveur SAHELYS)
- ☐ Git
- ☐ Un éditeur de code (VS Code recommandé, avec extension Python)
- ☐ Postman ou simplement `/docs` de FastAPI pour tester les endpoints

4.2 3.2 Installation des dépendances Python

```
# Créer l'environnement virtuel
python -m venv venv
source venv/bin/activate      # sous Linux/Mac
venv\Scripts\activate        # sous Windows

# Installer les dépendances principales
pip install fastapi "uvicorn[standard]"
pip install sqlalchemy asyncpg alembic
pip install pyjwt "passlib[bcrypt]"
pip install python-dotenv httpx
pip install pytest
```

4.3 3.3 Vérification côté serveur SAHELYS (à faire avec la direction/IT)

- ☐ Confirmer la présence ou non d'un GPU sur le serveur
- ☐ Confirmer la RAM disponible
- ☐ Obtenir les accès (SSH ou autre) pour installer Ollama
- ☐ Installer Ollama sur le serveur : `curl -fsSL https://ollama.com/install.sh | sh`
- ☐ Télécharger le modèle IA choisi selon la capacité réelle du serveur — par exemple `ollama pull mistral` (7B, léger) si le serveur a des ressources limitées, ou un modèle Mistral plus puissant (ex. Mistral Small, Mistral Medium) si le serveur le permet ; le choix final se fait après vérification RAM/GPU (voir section 3.3)

- ☐ Vérifier que PostgreSQL est installé avec l'extension pgvector activable (`CREATE EXTENSION vector;`)

4.4 3.4 Fichier .env — un par service (jamais commité dans Git)

backend/.env

```
DATABASE_URL=postgresql+asyncpg://utilisateur:motdepasse@localhost:5432/sahelys
SECRET_KEY=<clé secrète longue et aléatoire>
JWT_ALGORITHM=HS256
JWT_EXPIRATION_MINUTES=30
IA_SERVICE_URL=http://localhost:8001
```

ia/.env

```
OLLAMA_URL=http://localhost:11434
OLLAMA_MODEL=mistral
```

4.5 3.5 Lancer les deux services en développement

```
# Terminal 1 - service backend
cd backend && uvicorn main:app --reload --port 8000

# Terminal 2 - service IA
cd ia && uvicorn main:app --reload --port 8001
```

5 4. Architecture du système

flowchart LR

```
U[Collaborateur / Référent / Administrateur] -->|HTTPS| FE[Frontend<br/>Angular + Tailwind]
FE -->|API REST + JWT| BE[Service Backend<br/>FastAPI · port 8000]
BE -->|question| RAG[Module RAG<br/>YANOGO Fabiana]
RAG -->|recherche vectorielle| DB[(PostgreSQL<br/>+ pgvector)]
BE -->|prompt + contexte, HTTP| IA[Service IA<br/>FastAPI · port 8001]
IA -->|appel modèle| OLLAMA[Mistral via Ollama<br/>on-premise]
OLLAMA -->|réponse en streaming| IA
IA -->|réponse en streaming| BE
BE -->|réponse en streaming + sources| FE
BE -->|logs, agents, utilisateurs| DB
```

Changement de stack frontend : Next.js → Angular, décision d'équipe du 17 août 2026 (Personne C), soumise à avenant formel au cahier des charges (§9) — validé par les maîtres de stage. N'affecte pas mon bloc — le contrat d'API (JSON/streaming) reste identique quel que soit le framework frontend.

Backend et IA en deux services séparés : le service Backend ne connaît plus directement Ollama — il appelle le service IA en HTTP, qui lui-même appelle Ollama.

Trois maillons de streaming à la suite (Ollama → service IA → service Backend → Frontend), donc trois points où une erreur peut survenir (voir section 6, gestion d'erreurs).

Principe central du cahier des charges : **le modèle d'IA n'est jamais la source de vérité** — il répond uniquement à partir des passages retournés par le module RAG.

(Diagrammes de séquence détaillés par flux — authentification, question, création d'agent, association de document — disponibles dans le document de modélisation séparé.)

6 5. Modèle de données

```
erDiagram
    UTILISATEUR {
        uuid id
        string nom
        string direction
        string role
        string mot_de_passe_hash
    }
    AGENT {
        uuid id
        string nom
        string description
        text instructions
        string direction
        string statut
    }
    DOCUMENT {
        int id
        string nom
        string type
        string niveau_confidentialite
        string statut_indexation
        string chemin_fichier
        uuid agent_id
        timestamp date_ajout
    }
    LOG_INTERACTION {
        uuid id
        uuid utilisateur_id
        uuid agent_id
        text question
        text reponse
        text sources_citees
        timestamp date_heure
    }
```

```

AUTORISATION {
    uuid utilisateur_id
    uuid agent_id
}

AGENT ||--o{ DOCUMENT : "utilise"
UTILISATEUR ||--o{ LOG_INTERACTION : "effectue"
AGENT ||--o{ LOG_INTERACTION : "concerne"
UTILISATEUR ||--o{ AUTORISATION : "a"
AGENT ||--o{ AUTORISATION : "autorisé via"

```

Point de jonction avec le module RAG (YANOGO Fabiana) : sa table de découpage/vecteurs (CHUNK) référence `DOCUMENT.id` (entier, `SERIAL`).

Table DOCUMENT : schéma et responsabilités finalisés avec YANOGO Fabiana

- La table est **créée et migrée par le bloc RAG** (Fabiana) — le Backend ne la crée pas via sa propre migration Alembic, pour éviter que les deux blocs définissent chacun leur version de la table.
- **Insertion de la ligne** : le Backend insère la ligne initiale dès réception de l'upload (`type`, `niveau_confidentialite` donnés par le référent, `statut_indexation` = 'en_attente', `chemin_fichier` une fois le fichier sauvegardé), puis transmet le fichier au module RAG pour indexation.
- **Mise à jour du statut** : le module RAG met à jour `statut_indexation` directement dans la table pendant son pipeline (`en_cours` → `indexe/erreur`) ; le Backend le lit directement en base pour informer le référent — pas de notification/callback entre les deux blocs, lecture/écriture directe sur la table partagée.

Valeurs figées, décisions d'équipe du 17 août 2026 :

- `AGENT.statut` : brouillon / publié / désactivé
- `DOCUMENT.statut_indexation` : `en_attente` / `en_cours` / `indexe` / `erreur` (sans accent, pour éviter tout désaccord d'encodage entre les deux blocs) — protégé par une contrainte `CHECK` directement sur la colonne, ajoutée par YANOGO Fabiana dans son script de création de table :

```

statut_indexation TEXT DEFAULT 'en_attente'
CHECK (statut_indexation IN ('en_attente', 'en_cours', 'indexe', 'erreur'))

```

Pas de sous-catégories d'erreur (extraction vs embedding) en Phase 1 — le référent a seulement besoin de savoir qu'il y a un problème, pas le détail technique ; à réévaluer en Phase 2 si le besoin se confirme.

- Le champ **reference** (repère de citation type "Article 3" ou "Page 4") vit **côté RAG**, dans les chunks retournés — pas dans la table `DOCUMENT` elle-même

Encore ouvert, non tranché avec l'équipe à date : l'emplacement exact du dossier de stockage des fichiers (`chemin_fichier`) — recommandation : dossier local sur le serveur SAHELAYS (ex. `/var/sahelys/documents/`), cohérent avec la contrainte de souveraineté (§3.1), à confirmer en équipe. Le format d'upload (fichier direct vs métadonnées séparées) reste ouvert également — recommandation : upload direct en une seule action.

7 6. Sécurité et authentification

- **Librairie** : PyJWT, algorithme HS256
- **Mot de passe** : jamais stocké en clair, hachage bcrypt/argon2 via passlib
- **Contenu du token JWT** : `id utilisateur`, `direction`, `rôle`, `expiration` — volontairement **minimal**, ne contient **pas** la liste des agents autorisés (voir note ci-dessous)
- **Durée de vie** : token d'accès court (15-30 min) ; refresh token à évaluer selon le temps disponible
- **Secrets** : dans `.env`, jamais dans le code
- **Transport** : HTTPS obligatoire en production
- **Rôles (3, depuis la décision d'équipe du 17 août 2026)** :
 - **collaborateur** : consultation/question uniquement
 - **réfèrent** : gestion des agents et documents de sa propre direction
 - **administrateur** : supervision de toute la plateforme, toutes directions confondues (voir endpoints §7)

Pourquoi la liste des agents autorisés n'est pas dans le token : NANA Marc (frontend) avait proposé de l'y inclure. Risque identifié : le token vit 15-30 min ; si un réfèrent retire un accès pendant ce délai, le token reste valide avec l'ancienne liste jusqu'à expiration. Le frontend doit donc appeler `GET /agents` pour obtenir la liste à jour, vérifiée en base à chaque requête — pas lue depuis le token. Point à confirmer avec NANA Marc.

Conformité aux règles de sécurité du pilote (§7 du cahier des charges) :

Règle	Couverte par
Authentification obligatoire	JWT sur tous les endpoints sauf <code>/auth/login</code>
Accès limité à la direction RH (pilote)	Champ <code>direction</code> dans le token + filtrage
Aucune action d'écriture externe	Endpoints limités à la consultation et à la gestion interne des agents
Citation des sources	Champ <code>sources</code> dans chaque réponse
Journalisation	Table <code>LOG_INTERACTION</code>

8 7. Endpoints à construire

Méthode	Endpoint	Rôle requis	Description
POST	<code>/auth/login</code>	—	Authentification, retourne le JWT
GET	<code>/agents</code>	collaborateur	Liste les agents accessibles
POST	<code>/agents</code>	réfèrent	Crée un agent
PUT	<code>/agents/{id}</code>	réfèrent	Modifie un agent
DELETE	<code>/agents/{id}</code>	réfèrent	Supprime/désactive un agent
POST	<code>/agents/{id}/documents</code>	réfèrent	Associe un document à un agent

Méthode	Endpoint	Rôle requis	Description
POST	/agents/{id}/query	collaborateur	Pose une question à l'agent — réponse en streaming (décision d'équipe du 17 août 2026), sources envoyées en dernier événement du flux
GET	/admin/system/dashboard	administrateur	Vue globale : nb directions, agents, utilisateurs actifs
GET/POST/PUT	/admin/system/users	administrateur	Gestion des comptes, attribution des rôles
GET	/admin/system/logs	administrateur	Logs toutes directions confondues (vs référent, limité à la sienne)

Note sur /admin/system/settings proposé par NANA Marc : périmètre flou, non appuyé par la note conceptuelle. À ne construire qu'après clarification avec des exemples concrets — ne pas l'ajouter à ce tableau tant que ce n'est pas précisé.

8.1 Endpoints internes du service IA (port 8001, jamais exposés au frontend)

Méthode	Endpoint	Description
POST	/generate	Reçoit {question, contexte}, appelle Ollama, retourne la réponse en streaming
GET	/health	Vérifie que le service IA (et Ollama) répond — utilisé par le service Backend pour détecter une panne avant d'envoyer une requête

9 8. Plan de travail par sprints

Pas de dates — chaque sprint se termine quand ses tâches sont cochées. Un sprint dépend en général du précédent.

9.1 Sprint 0 — Préparation de l'environnement

- ☐ Installer Python, PostgreSQL client, Git, VS Code
- ☐ Créer l'environnement virtuel et installer les dépendances (section 3.2)
- ☐ Obtenir les accès au serveur SAHELYS
- ☐ Vérifier la capacité serveur (RAM/GPU) avec la direction
- ☐ Installer Ollama et le modèle Mistral sur le serveur
- ☐ Créer le fichier `.env`
- ☐ Vérifier l'accès à la base PostgreSQL et activer l'extension `pgvector`

Livrable du sprint : environnement de développement fonctionnel, projet FastAPI qui démarre (`uvicorn main:app --reload`) et répond sur `/docs`.

9.2 Sprint 1 — Squelette du projet et modèle de données

- ☐ Structurer le projet FastAPI (dossiers : routes, modèles, services, config)

- ☐ Définir les modèles SQLAlchemy : `Utilisateur`, `Agent`, `LogInteraction`, `Autorisation`
- ☐ Modèle SQLAlchemy `Document` en **lecture/écriture uniquement** (table créée par la migration du bloc RAG, pas par ma propre migration Alembic)
- ☐ Configurer Alembic et générer la première migration (sur mes propres tables)
- ☐ Créer les tables en base, vérifier la coordination avec YANOGO Fabiana pour `Document/Chunk`

Livrable du sprint : tables créées en base, vérifiables via `psql` ou `pgAdmin`.

9.3 Sprint 2 — Authentification

- ☐ Endpoint `POST /auth/login`
- ☐ Hachage et vérification des mots de passe (`passlib/bcrypt`)
- ☐ Génération du JWT (`PyJWT`, payload : `id`, `direction`, `rôle`, `expiration`)
- ☐ Middleware/dépendance `FastAPI` pour vérifier le token sur les routes protégées
- ☐ Gestion des rôles (`collaborateur` vs `réfèrent`) dans la vérification

Livrable du sprint : un utilisateur peut se connecter et obtenir un token valide ; un token invalide ou absent est rejeté (401).

9.4 Sprint 3 — Gestion des agents (CRUD)

- ☐ `POST /agents` (réservé au rôle référent)
- ☐ `GET /agents` (filtré selon la direction/autorisation de l'utilisateur)
- ☐ `PUT /agents/{id}`
- ☐ `DELETE /agents/{id}`
- ☐ Vérification des permissions sur chaque endpoint

Livrable du sprint : un référent peut créer, modifier, lister et supprimer un agent via l'API, testable dans `/docs`.

9.5 Sprint 4 — Service IA séparé (Mistral/Ollama)

- ☐ Créer l'application `FastAPI` du service IA (`ia/main.py`, port 8001)
- ☐ Endpoint interne `POST /generate` : reçoit question + contexte, appelle Ollama, retourne en streaming
- ☐ Endpoint `GET /health` pour vérifier que le service (et Ollama) répond
- ☐ Fonction d'appel HTTP vers Ollama (via `httpx`) depuis ce service
- ☐ Construction du prompt (question + contexte)
- ☐ Gestion du cas où Ollama ne répond pas (timeout, erreur 500) — remontée proprement au service Backend
- ☐ Côté service Backend : fonction cliente qui appelle `IA_SERVICE_URL/generate` et gère le cas où le service IA lui-même est injoignable
- ☐ Test manuel : Backend → service IA → Ollama → réponse, sans encore le RAG

Livrable du sprint : deux services `FastAPI` qui tournent indépendamment (ports 8000 et 8001), le Backend peut obtenir une réponse générée en passant par le service IA.

9.6 Sprint 5 — Intégration avec le module RAG (YANOGO Fabiana) + streaming

- ☐ Fixer avec YANOGO Fabiana le format exact d'échange : `{"passages": [{"texte": "...", "document": "...", "reference": "...", "score": ...}]}`
- ☐ Implémenter l'appel du backend vers le module RAG
- ☐ Gérer le cas "aucun passage pertinent trouvé"
- ☐ Endpoint `POST /agents/{id}/query` en **streaming** (Server-Sent Events ou `StreamingResponse` FastAPI) : question → RAG → Mistral → tokens envoyés au fil de l'eau → sources envoyées en dernier événement
- ☐ Accumuler la réponse complète côté serveur pendant le streaming (nécessaire pour la journalisation, Sprint 7)

Livrable du sprint : une question posée à un agent retourne une réponse sourcée en streaming, construite à partir des documents réels.

9.7 Sprint 6 — Association de documents

- ☐ `POST /agents/{id}/documents` (réservé au rôle référent)
- ☐ Sauvegarde du fichier physique sur le serveur (`chemin_fichier`)
- ☐ Insertion de la ligne dans `Document` (table gérée par YANOGO Fabiana) : `type`, `niveau_confidentialite`, `statut_indexation = 'en_attente'`, `chemin_fichier`, `agent_id`
- ☐ Transmission du document au module RAG pour indexation
- ☐ Lecture de `statut_indexation` directement en base (mis à jour par le RAG) pour informer le référent de l'avancement

Livrable du sprint : un référent peut uploader un document, la ligne est créée par le Backend, et le statut d'indexation évolue visiblement au fur et à mesure que le RAG traite le fichier.

9.8 Sprint 7 — Journalisation

- ☐ Enregistrement automatique de chaque question/réponse dans `LogInteraction`
- ☐ Champs enregistrés : utilisateur, agent, question, réponse, sources, date/heure
- ☐ Vérification que le log est bien écrit à chaque appel de `/agents/{id}/query`

Livrable du sprint : chaque interaction est traçable en base, conforme à l'exigence de sécurité §7.

9.9 Sprint 8 — Rôle administrateur central

- ☐ Ajouter le rôle `administrateur` à la vérification JWT (3e rôle, en plus de collaborateur/référent)
- ☐ `GET /admin/system/dashboard` — agrégats toutes directions
- ☐ `GET/POST/PUT /admin/system/users` — gestion des comptes et attribution des rôles

- ☐ GET `/admin/system/logs` — logs toutes directions (vs référent, limité à la sienne)
- ☐ Vérifier que `/admin/system/*` est bien inaccessible aux rôles collaborateur et référent (403)

Livable du sprint : un administrateur peut superviser l'ensemble de la plateforme, indépendamment des directions.

9.10 Sprint 9 — Intégration avec l'équipe

- ☐ Test de bout en bout avec le frontend de NANA Marc (portail de chat + interface no-code)
- ☐ Test de bout en bout avec le module RAG réel de YANOGO Fabiana (pas de données factices)
- ☐ Résolution des écarts de format entre les trois blocs
- ☐ Vérification des cas d'erreur (401, 403, 404, 500) côté frontend

Livable du sprint : l'agent pilote RH fonctionne de bout en bout, comme prévu au §6 du cahier des charges.

9.11 Sprint 10 — Tests et préparation de la démonstration

- ☐ Tester une série de questions RH/IT réelles
- ☐ Corriger les cas de réponses incorrectes ou mal sourcées
- ☐ Vérifier tous les endpoints via `/docs` une dernière fois
- ☐ Préparer les éléments de démonstration côté backend (exemples de logs, exemples de réponses)

Livable du sprint : backend prêt pour la démonstration finale de la Phase 1.

10 9. État d'avancement global

Sprint	Statut
Sprint 0 — Préparation environnement	☐ Non commencé
Sprint 1 — Squelette + modèle de données	☐ Non commencé
Sprint 2 — Authentification	☐ Non commencé
Sprint 3 — Gestion des agents (CRUD)	☐ Non commencé
Sprint 4 — Connexion Mistral/Ollama	☐ Non commencé
Sprint 5 — Intégration RAG	☐ Non commencé
Sprint 6 — Association de documents	☐ Non commencé
Sprint 7 — Journalisation	☐ Non commencé
Sprint 8 — Rôle administrateur central	☐ Non commencé
Sprint 9 — Intégration équipe	☐ Non commencé
Sprint 10 — Tests et démonstration	☐ Non commencé

À mettre à jour au fur et à mesure ([] Non commencé / [~] En cours / [x] Terminé).

Document de référence unique pour le développement backend, Phase 1 uniquement. S'appuie sur le cahier des charges officiel du 14 août 2026 et la note conceptuelle du projet. Un nouveau document sera créé pour la Phase 2.